

实验七 编码器信号处理

实验八 PID 闭环控制

实验七 编码器信号处理

7.1 实验目的

- 掌握 GMR 磁编码器 AB 相正交脉冲的基本测速原理。
- 理解编码器 A、B 两相信号相位差与电机转向判断方法。
- 结合本项目硬件连接，完成四路电机编码器信号采集。
- 能够在 OLED 上实时显示四个电机的编码器检测速度，为后续 PID 闭环控制提供反馈量。

7.2 实验简介

7.2.1 GMR 磁编码器工作原理

小车使用的 MG370 电机带有 GMR 磁编码器。电机转动时，磁编码器输出 A、B 两路方波信号，两路信号相位相差约 90° ，因此也称为正交编码器信号。通过统计单位时间内的脉冲数量可以得到电机转速，通过判断 A 相变化时 B 相的电平状态可以判断电机转动方向。

理论上，STM32 定时器可以配置为正交解码模式，由硬件自动对 AB 相边沿进行计数。PDF 示例中使用 TIM_EncoderMode_TI12、输入捕获滤波和定时读取计数值的方式获得编码器速度。对于本项目的小车，由于编码器线较长、电机干扰较明显，演示工程采用 TIM5 周期采样 GPIO 的方式完成较稳健的 2 倍频解码，便于先验证编码器反馈是否可靠。



图 7.1 GMR 磁编码器 AB 相正交脉冲示意图

7.2.2 编码器引脚对应关系

结合主板原理图和当前 PID 工程，四路编码器输入引脚分配如下表所示。M3、M4 位于车体右侧，安装方向与左侧电机相反，因此在软件中通过极性系数进行方向统一。

电机编号	A 相输入	B 相输入	软件方向极性
M1	PA8	PA9	+1
M2	PD12	PD13	+1
M3	PA7	PA6	-1
M4	PC6	PC7	-1

表 7.1 四路编码器引脚分配表

7.2.3 编码器测速的软件处理流程

本实验的测速流程为：先将四组 AB 相引脚配置为上拉输入，再使用 TIM5 产生 100us 周期中断，在中断中读取每路 A、B 电平。当检测到 A 相发生跳变时，根据 A 与 B 的关系判断本次脉冲方向，并累加到对应电机的脉冲计数变量中。主循环或 PID 控制定时器每 50ms 调用一次 ENCODER_Update()，把累计脉冲转换为本周速度值。

```
#define ENCODER_M1_POLARITY      1
#define ENCODER_M2_POLARITY      1
#define ENCODER_M3_POLARITY     -1
#define ENCODER_M4_POLARITY     -1
#define ENCODER_INPUT_FILTER     15
#define ENCODER_COUNTER_PERIOD   65535
#define ENCODER_PID_SCALE_DIV    100

static volatile int16_t g_encoder_raw_pulse[ENCODER_MOTOR_COUNT] = {0, 0, 0, 0};
static volatile int16_t g_encoder_speed_pulse[ENCODER_MOTOR_COUNT] = {0, 0, 0, 0};
static volatile uint8_t g_encoder_zero_count[ENCODER_MOTOR_COUNT] = {0, 0, 0, 0};
```

其中 ENCODER_SAMPLE_MS 表示速度刷新周期，g_encoder_raw_pulse 表示本周原始脉冲数，g_encoder_speed_pulse 表示滤波后的速度反馈量。由于本工程采用 A 相双边沿计数，因此属于 2 倍频解码；若后续改用定时器正交解码 TI12 模式，则一般可获得 4 倍频计数。

7.3 实验步骤

7.3.1 新建编码器驱动文件

在 HARDWARE/MOTOR 目录下建立 encoder.c 和 encoder.h，并在 Keil 工程中加入源文件。头文件中定义电机数量、测速周期和单圈脉冲数。单圈脉冲数需要结合实际编码器线

数与减速比标定，当前工程默认值为 1560。

```
1 #ifndef __ENCODER_H
2 #define __ENCODER_H
3
4 #include "stm32f4xx.h"
5 #include <stdint.h>
6
7 #define ENCODER_MOTOR_COUNT      4
8 #define ENCODER_SAMPLE_MS       20
9
10 /*
11  * Calibrate this value with the real motor.
12  * Usually: encoder lines * gear ratio * 4 for AB quadrature counting.
13  */
14 #define ENCODER_PULSES_PER_REV   1560
15
16 void ENCODER_Init(void);
17 void ENCODER_ResetAll(void);
18 void ENCODER_Update(void);
19 int16_t ENCODER_GetRawPulse(uint8_t motor);
20 int16_t ENCODER_GetSpeedPulse(uint8_t motor);
21 int16_t ENCODER_GetRPM(uint8_t motor);
22 uint8_t ENCODER_GetABState(uint8_t motor);
23
24 #endif
25
```

7.3.2 初始化编码器 GPIO 输入

四路编码器信号均配置为 GPIO 上拉输入。上拉输入可以避免线缆悬空时出现不确定电平，提高低速手拨测试时的稳定性。

```
static void ENCODER_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;

    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA |
                           RCC_AHB1Periph_GPIOC |
                           RCC_AHB1Periph_GPIOD, ENABLE);

    GPIO_PinAFConfig(GPIOA, GPIO_PinSource8, GPIO_AF_TIM1);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource9, GPIO_AF_TIM1);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource6, GPIO_AF_TIM3);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource7, GPIO_AF_TIM3);
    GPIO_PinAFConfig(GPIOD, GPIO_PinSource12, GPIO_AF_TIM4);
    GPIO_PinAFConfig(GPIOD, GPIO_PinSource13, GPIO_AF_TIM4);
    GPIO_PinAFConfig(GPIOC, GPIO_PinSource6, GPIO_AF_TIM8);
    GPIO_PinAFConfig(GPIOC, GPIO_PinSource7, GPIO_AF_TIM8);

    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_2MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7 |
                                   GPIO_Pin_8 | GPIO_Pin_9;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12 | GPIO_Pin_13;

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6 | GPIO_Pin_7;
    GPIO_Init(GPIOC, &GPIO_InitStructure);
}
```

7.3.3 配置 TIM5 周期采样

TIM5 挂载在 APB1 总线上，系统时钟为 168MHz 时，APB1 定时器时钟为 84MHz。将预分频设置为 84-1 后计数频率为 1MHz，周期设置为 ENCODER_POLL_US-1，即可得到 100us 采样周期。

```
static void ENCODER_Timer_Init(TIM_TypeDef *tim)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    TIM_ICInitTypeDef TIM_ICInitStructure;

    ENCODER_Timer_Reset(tim);

    TIM_TimeBaseStructInit(&TIM_TimeBaseStructure);
    TIM_TimeBaseStructure.TIM_Period = ENCODER_COUNTER_PERIOD;
    TIM_TimeBaseStructure.TIM_Prescaler = 0;
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV4;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(tim, &TIM_TimeBaseStructure);

    TIM_EncoderInterfaceConfig(tim,
                               TIM_EncoderMode_TI12,
                               TIM_ICPolarity_Rising,
                               TIM_ICPolarity_Rising);

    TIM_ICStructInit(&TIM_ICInitStructure);
    TIM_ICInitStructure.TIM_ICFilter = ENCODER_INPUT_FILTER;
    TIM_ICInitStructure.TIM_ICPrescaler = TIM_ICPSC_DIV1;

    TIM_ICInitStructure.TIM_Channel = TIM_Channel_1;
    TIM_ICInit(tim, &TIM_ICInitStructure);

    TIM_SetCounter(tim, 0);
    TIM_ClearITPendingBit(tim, TIM_IT_Update);
    TIM_ITConfig(tim, TIM_IT_Update, DISABLE);
    TIM_Cmd(tim, ENABLE);
}
```

图 7.2 TIM5 定时采样编码器信号的软件流程

7.3.4 编写 AB 相信号读取和方向判断函数

每路电机读取 A、B 两相信号后，只在 A 相发生变化时计数。若 A 与 B 电平不同，则认为本次为正方向；若 A 与 B 电平相同，则认为本次为反方向。再乘以对应电机的软件极性，保证 OLED 上显示的速度方向一致。

```

static uint8_t ENCODER_ReadAB(uint8_t motor)
{
    uint8_t a;
    uint8_t b;

    switch (motor)
    {
        case 1:
            a = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_8);
            b = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_9);
            break;

        case 2:
            a = GPIO_ReadInputDataBit(GPIOD, GPIO_Pin_12);
            b = GPIO_ReadInputDataBit(GPIOD, GPIO_Pin_13);
            break;

        case 3:
            a = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_7);
            b = GPIO_ReadInputDataBit(GPIOA, GPIO_Pin_6);
            break;

        case 4:
            a = GPIO_ReadInputDataBit(GPIOC, GPIO_Pin_6);
            b = GPIO_ReadInputDataBit(GPIOC, GPIO_Pin_7);
            break;

        break,

        default:
            return 0;
    }

    return (uint8_t)((a << 1) | b);
}

```

`g_encoder_guard` 用于简单滤除过窄抖动脉冲。若电机稍快时数值异常飘高，可适当增大 `ENCODER_MIN_EDGE_TICKS`；若手拨速度较快时偶尔漏计，可适当减小该值或改用硬件正交解码并增加输入滤波。

7.3.5 编写速度更新函数

`ENCODER_Update()` 在固定周期内读取累计脉冲并清零，然后对速度值做一次简单平均滤波。OLED 显示和 PID 控制都读取 `ENCODER_GetSpeedPulse()`，这样可以降低显示跳动和控制抖动。

```

void ENCODER_Update(void)
{
    int16_t raw[ENCODER_MOTOR_COUNT];
    int16_t speed;
    int32_t pulse;
    uint8_t i;

    raw[0] = ENCODER_ReadTimer(TIM1);
    raw[1] = ENCODER_ReadTimer(TIM4);
    raw[2] = ENCODER_ReadTimer(TIM3);
    raw[3] = ENCODER_ReadTimer(TIM8);

    for (i = 0; i < ENCODER_MOTOR_COUNT; i++)
    {
        pulse = (int32_t)raw[i] * ENCODER_GetPolarity(i);
        g_encoder_raw_pulse[i] = ENCODER_LimitInt16(pulse);
        speed = ENCODER_ToPidSpeed(pulse);

        if (speed == 0)
        {
            if (g_encoder_zero_count[i] < 255)
            {
                g_encoder_zero_count[i]++;
            }

            if ((g_encoder_speed_pulse[i] > 0) && (g_encoder_zero_count[i] <= 5))
            {
                g_encoder_speed_pulse[i]--;
            }
            else
            {
                g_encoder_speed_pulse[i] = 0;
            }
        }
        else
        {
            g_encoder_zero_count[i] = 0;
            g_encoder_speed_pulse[i] =
                (int16_t)((g_encoder_speed_pulse[i] * 2 + speed + 1) / 3);
        }
    }
}

```

7.3.6 OLED 显示四路编码器速度

为了便于验证四个编码器是否接线正确，可以先不启动电机，仅手拨车轮并观察 OLED 数值变化。若拨动某个车轮时对应通道有正负变化，且停止后数值逐渐回到 0，说明该路编码器采集基本正常。由于经常使用 oled，故 oled 屏幕代码此次自行设计。

7.4 实验现象与调试方法

- 手拨 M1、M2、M3、M4，OLED 对应通道应分别变化，未拨动的通道应接近 0。
- 若某一路始终为 0，应重点检查该路 A、B 相接线、电源、GND 以及 GPIO 引脚定义。
- 若某一路方向为负，可交换 A、B 相，或在 ENCODER_GetPolarity() 中修改该路极性。

- 若轻微拨动就出现很大的速度值，应检查编码器线是否靠近电机电源线，并适当增大软件滤波或硬件输入滤波。
- 四个电机实际速度不完全一致是正常现象，原因包括减速箱摩擦、电机个体差异、轮胎负载和电池电压变化。

实验八 PID 闭环控制

8.1 实验目的

- 理解开环 PWM 调速与闭环编码器调速的区别。
- 掌握 PID 控制中比例、积分、微分三个环节的作用。
- 能够利用编码器速度反馈自动修正四路电机 PWM 占空比。
- 能够通过 OLED 显示目标速度、实际速度、PWM 输出和误差，观察 PID 调控效果。
- 能够通过按键启动、停止 PID，并调节目标速度完成课堂演示。

8.2 实验简介

8.2.1 为什么需要 PID 闭环控制

在实验六中，四路电机可以通过 PWM 独立调速。但开环 PWM 只能给定电机驱动占空比，不能保证四个车轮速度相同。实际小车运行时，不同电机的减速箱摩擦、轮胎阻力、左右安装方向以及地面负载都会造成速度差异，最终表现为小车跑偏、抖动或低速无法平稳起步。

PID 闭环控制的核心思想是：不断读取编码器实际速度，将目标速度与实际速度作差，根据误差动态调节 PWM 占空比。速度偏低时提高占空比，速度偏高时降低占空比，从而让四个车轮趋近同一目标速度。

其中： K_p —— 控制器的比例系数

T_i —— 控制器的积分时间，也称积分系数

T_d —— 控制器的微分时间，也称微分系数

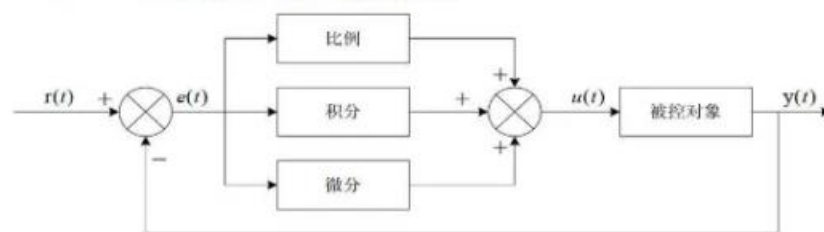


图 8.1 电机速度闭环控制框图

8.2.2 PID 控制原理

PID 控制由 P、I、D 三个部分组成。比例 P 根据当前误差立即修正输出，P 越大响应越快，但过大容易振荡；积分 I 根据误差累积消除静差，I 太大容易造成超调和恢复慢；微分 D 根据误差变化趋势进行抑制，可以减小抖动，但过大时会放大测量噪声。

本项目为了便于在单片机中计算，将 PID 参数扩大 10 倍保存。例如 $KP_X10=3$ 表示

$K_p=0.3$ 。控制器每 50ms 运行一次，目标值和实际值均使用“每 50ms 的编码器脉冲数”，因此 OLED 上的 T、A、M1-M4 数值较小是正常的，它们不是 PWM 占空比，也不是直接的 RPM。

```
error = target - measured;

integral += error;

output = base_duty + target / 2

        + (kp * error + ki * integral + kd * (error - last_error)) / 10;
```

8.2.3 本项目中的 PID 控制方案

当前工程将编码器采集、PID 计算和 OLED 演示分成三个层次：encoder.c 负责四路编码器采集，speed_control.c 负责速度闭环控制，main.c 负责 OLED 显示和按键交互。

- KEY1：启动或停止 PID 闭环控制。
- KEY2：目标速度递增，达到上限后回到最小值。
- OLED 第一行：显示 PID 状态、目标速度 T 和四轮平均速度 A。
- OLED 第二行：显示左侧、右侧平均 PWM 占空比和速度误差 E。
- OLED 第三、四行：显示 M1-M4 的速度反馈和对应 PWM 输出。

8.3 实验步骤

8.3.1 新建速度闭环控制文件

在 HARDWARE/MOTOR 目录下建立 speed_control.c 和 speed_control.h，并在 Keil 工程中加入源文件。该模块对外提供初始化、启动、停止、设置目标速度、设置 PID 参数和读取显示数据等接口。

```
#ifndef __SPEED_CTRL_H
#define __SPEED_CTRL_H

#include "stm32f4xx.h"
#include "encoder.h"
#include <stdint.h>

#define SPEED_CTRL_PERIOD_MS      ENCODER_SAMPLE_MS
#define SPEED_CTRL_MAX_TARGET_PULSE 200
#define SPEED_CTRL_PID_SCALE_VALUE 100

void SPEED_CTRL_Init(void);
void SPEED_CTRL_Start(void);
void SPEED_CTRL_Stop(void);
void SPEED_CTRL_SetTargetPulse(int16_t pulse);
void SPEED_CTRL_SetTargetRPM(int16_t rpm);
void SPEED_CTRL_SetPID(uint8_t motor, int16_t kp_scaled, int16_t ki_scaled, int16_t kd_scaled);
void SPEED_CTRL_Task(void);
int16_t SPEED_CTRL_GetTargetPulse(void);
int16_t SPEED_CTRL_GetSpeedPulse(uint8_t motor);
uint16_t SPEED_CTRL_GetDuty(uint8_t motor);

#endif
```

8.3.2 配置 TIM7 作为 PID 控制周期

TIM7 用作 PID 控制定时器。它以 SPEED_CTRL_PERIOD_MS 为周期进入中断，在中断服

务函数或定时任务中调用 `SPEED_CTRL_Task()`。控制周期应与 `ENCODER_SAMPLE_MS` 保持一致。

```
static void SPEED_CTRL_TIM7_Init(void)
{
    TIM_TimeBaseInitTypeDef TIM_TimeBaseStructure;
    NVIC_InitTypeDef NVIC_InitStructure;

    RCC_APB1PeriphClockCmd(RCC_APB1Periph_TIM7, ENABLE);

    TIM_TimeBaseStructure.TIM_Period = SPEED_CTRL_PERIOD_MS * 1000 - 1;
    TIM_TimeBaseStructure.TIM_Prescaler = 84 - 1;
    TIM_TimeBaseStructure.TIM_ClockDivision = TIM_CKD_DIV1;
    TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
    TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;
    TIM_TimeBaseInit(TIM7, &TIM_TimeBaseStructure);

    TIM_ClearITPendingBit(TIM7, TIM_IT_Update);
    TIM_ITConfig(TIM7, TIM_IT_Update, ENABLE);

    NVIC_InitStructure.NVIC_IRQChannel = TIM7_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 3;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);

    TIM_Cmd(TIM7, ENABLE);
}
```

8.3.3 编写 PID 结构体与初始化函数

每个电机对应一个 PID 结构体，分别保存参数、目标值、误差、积分项和输出 PWM。四个电机独立计算，便于补偿电机个体差异。

```
#define SPEED_CTRL_PID_SCALE          SPEED_CTRL_PID_SCALE_VALUE
#define SPEED_CTRL_DEFAULT_KP         *
#define SPEED_CTRL_DEFAULT_KI         *
#define SPEED_CTRL_DEFAULT_KD         *
#define SPEED_CTRL_MIN_DUTY           0
#define SPEED_CTRL_MAX_DUTY           30
#define SPEED_CTRL_TARGET_STEP         4
#define SPEED_CTRL_FEED_FORWARD_DIV   2
#define SPEED_CTRL_INTEGRAL_LIMIT      100
#define SPEED_CTRL_ERROR_DEADBAND      2
#define SPEED_CTRL_CORR_LIMIT          25
#define SPEED_CTRL_DUTY_STEP_LIMIT     2
#define SPEED_CTRL_LOW_TARGET_PULSE    10
```

（打*号表示自行取值）

8.3.4 编写 PID 计算函数

PID 输出并不直接从 0 开始计算，而是叠加了基础占空比和目标速度前馈。这样低速起步时电机更容易克服静摩擦，目标速度变化时响应也更快。M3、M4 由于右侧电机负载和安装方向差异，基础占空比与起步占空比略高。

```

static uint16_t SPEED_PID_Calc(uint8_t index, int16_t target, int16_t measured)
{
    SPEED_PID_T *pid;
    int16_t error_for_calc;
    int32_t base_output;
    int32_t correction;
    int32_t output;

    pid = &g_speed_pid[index];
    pid->target = target;
    pid->error = (int16_t)(pid->target - measured);

    if (pid->target == 0)
    {
        pid->integral = 0;
        pid->output = 0;
    }
    else
    {
        error_for_calc = pid->error;
        if (SPEED_Abs16(error_for_calc) <= SPEED_CTRL_ERROR_DEADBAND)
        {
            error_for_calc = 0;
        }

        pid->integral = (int16_t)(pid->integral + error_for_calc);
        pid->integral = SPEED_LimitInt16(pid->integral,
                                        -SPEED_CTRL_INTEGRAL_LIMIT,
                                        SPEED_CTRL_INTEGRAL_LIMIT);

        base_output = g_run_duty_base[index] +
                      (int32_t)pid->target / SPEED_CTRL_FEED_FORWARD_DIV;

        correction = ((int32_t)pid->kp * error_for_calc +
                      (int32_t)pid->ki * pid->integral +
                      (int32_t)pid->kd * (error_for_calc - pid->last_error)) /
                      SPEED_CTRL_PID_SCALE;
        correction = SPEED_LimitInt16((int16_t)correction,
                                        -SPEED_CTRL_CORR_LIMIT,
                                        SPEED_CTRL_CORR_LIMIT);

        output = base_output + correction;

        if ((measured <= 1) && (pid->error > 0) &&
            (output < g_breakaway_duty[index]))
        {
            output = g_breakaway_duty[index];
        }
        output = SPEED_LimitInt16((int16_t)output,
                                    SPEED_CTRL_MIN_DUTY,
                                    g_motor_max_duty[index]);

        g_motor_max_duty[index];

        if ((pid->target <= SPEED_CTRL_LOW_TARGET_PULSE) &&
            (output > g_low_target_max_duty[index]))
        {
            output = g_low_target_max_duty[index];
        }

        if (output > (int32_t)pid->output + SPEED_CTRL_DUTY_STEP_LIMIT)
        {
            output = (int32_t)pid->output + SPEED_CTRL_DUTY_STEP_LIMIT;
        }
        else if (output < (int32_t)pid->output - SPEED_CTRL_DUTY_STEP_LIMIT)
        {
            output = (int32_t)pid->output - SPEED_CTRL_DUTY_STEP_LIMIT;
        }

        pid->output = SPEED_LimitInt16((int16_t)output,
                                        SPEED_CTRL_MIN_DUTY,
                                        g_motor_max_duty[index]);
    }

    pid->prev_error = pid->last_error;
    pid->last_error = pid->error;

    return (uint16_t)pid->output;
}

```

输出结果需要限制在电机允许的占空比范围内,防止因编码器异常或参数过大导致 PWM 长时间满输出。

8.3.5 编写速度控制任务

SPEED_CTRL_Task() 是闭环控制的核心。函数先调用 ENCODER_Update() 刷新四路速度,再根据目标速度进行缓慢斜坡变化,最后对每个电机独立执行 PID 计算并更新 PWM。

```
void SPEED_CTRL_Task(void)
{
    uint8_t i;
    int16_t measured;
    uint16_t duty;

    ENCODER_Update();

    if (!g_speed_ctrl_enable)
    {
        return;
    }

    if (g_target_ramp < g_target_request)
    {
        g_target_ramp += SPEED_CTRL_TARGET_STEP;
        if (g_target_ramp > g_target_request)
        {
            g_target_ramp = g_target_request;
        }
    }
    else if (g_target_ramp > g_target_request)
    {
        g_target_ramp -= SPEED_CTRL_TARGET_STEP;
        if (g_target_ramp < g_target_request)
        {
            g_target_ramp = g_target_request;
        }
    }

    else if (g_target_ramp > g_target_request)
    {
        g_target_ramp -= SPEED_CTRL_TARGET_STEP;
        if (g_target_ramp < g_target_request)
        {
            g_target_ramp = g_target_request;
        }
    }

    for (i = 0; i < ENCODER_MOTOR_COUNT; i++)
    {
        measured = SPEED_Abs16(ENCODER_GetSpeedPulse(i + 1));
        duty = SPEED_PID_Calc(i, g_target_ramp, measured);
        g_motor_duty[i] = duty;
        MOTOR_SetDuty((uint8_t)(i + 1), duty);
    }
}

uint16_t SPEED_CTRL_GetTargetPulse(void)
```

图 8.2 PID 速度闭环控制软件流程

8.3.6 在 main.c 中设置 PID 参数

为了方便课堂调试，PID 参数统一放在 main.c 顶部宏定义中。修改 KP、KI、KD 后重新编译下载，即可观察 OLED 上速度变化和 PWM 输出变化。

```
/*
 * PID parameters can be tuned as decimals here.
 * Suggested start: Kp 0.20~0.50, Ki 0.03~0.10, Kd 0.00.
 */
#define MAIN_PID_KP          *
#define MAIN_PID_KI          *
#define MAIN_PID_KD          *
```

8.3.7 编写 OLED 显示界面

OLED 显示应同时展示目标速度、实际速度和 PWM 输出，这样学生可以直观看到 PID 的调控过程。例如目标 T 增大后，PWM 占空比会先升高，实际速度 A 和各路 M 值随后上升；当实际速度接近目标后，PWM 输出会自动回落到维持速度所需的占空比。（自行设计）

8.3.8 按键控制 PID 演示

KEY1 用于启动或停止 PID，KEY2 用于调节目标速度。演示时建议先将目标速度设为较小值，启动后观察四个电机能否平稳起步，再逐步增大目标速度。（自行设计）

8.4 PID 参数调试方法

PID 调试应在编码器读数稳定的基础上进行。如果编码器某一路读数为 0、方向相反或异常飘高，PID 会根据错误反馈调节 PWM，表现为速度不受控、某个轮子突然很快或目标变化没有效果。因此应先完成实验七的编码器验证，再进行实验八。

- 先将 KI、KD 设为 0，只调 KP。逐步增大 KP，使电机速度能够跟随目标，但不要出现明显振荡。
- 在 KP 基本合适后，少量增加 KI，用于减小长期误差。若速度恢复很慢或停不下来，说明积分过大。
- 若速度波动明显，可少量增加 KD 抑制变化，但编码器噪声较大时 KD 不宜过大。
- 若低目标速度下右侧电机一卡一卡，可适当提高右侧电机的 g_breakaway_duty 或 g_run_duty_base。
- 若目标速度超过某个值后速度不再提升，可能是电机已接近当前电池电压和负载下的最大速度，此时继续增大目标只会让 PWM 接近上限。

8.5 实验现象

下载程序后，OLED 显示 PID STOP，目标速度 T 为初始值，四路 PWM 输出为 0。按下

KEY2 可以调节目标速度，按下 KEY1 后进入 PID RUN 状态，四个电机开始转动。此时 OLED 上可以看到 A 和 M1-M4 逐渐接近 T，PWM 输出会根据每个电机实际速度自动变化。

当某个电机实际速度低于目标速度时，该电机对应的 PWM 输出会增大；当实际速度高于目标速度时，PWM 输出会减小。四个电机最终维持在接近目标速度的范围内，说明编码器反馈与 PID 调速闭环已经建立。

8.6 实验总结

本次实验在前面 PWM 电机驱动实验的基础上，加入了编码器速度反馈和 PID 闭环控制。实验七完成了四路编码器信号读取、方向统一、速度计算和 OLED 显示；实验八利用编码器速度作为反馈量，通过 PID 算法动态修正 PWM 占空比，实现四轮速度趋同和平稳运行。

通过本实验可以看到，闭环控制的前提是传感器反馈可靠。只有编码器读数方向正确、数值稳定，PID 参数调节才有意义。实际调试时应遵循“先验证编码器，再调低速起步，最后优化 PID 参数”的顺序。